MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

Mini-Project Report

Optimizing CUDA Image Convolution via Shared Memory Tiling and Separable Kernels

SUBMITTED BY

STUDENT NAME	REG. NO.
Vasishta Nandipati	230962095
Rishit Mathur	220962091
Krishiv Kolanu	230962023

Under the Guidance of:

Vidya Kamath

Assistant Professor

School of Computer Engineering

Manipal Institute of Technology, Manipal, Karnataka – 576104

ABSTRACT

Image convolution underpins a large class of image processing tasks ranging from classical spatial filtering to the convolutional layers of modern deep learning networks. Although GPU hardware offers significant parallelism, a straightforward CUDA implementation of 2-D convolution makes no effort to reuse data cached on-chip and ends up issuing the same global memory reads repeatedly, leaving most of the device bandwidth idle. This project addresses that inefficiency through two optimisations. The first loads a tile of input pixels, together with a border region of width equal to the kernel radius, into shared memory once per thread block; subsequent computation then reads entirely from on-chip storage. The second decomposes a separable 2-D kernel into a horizontal and a vertical 1-D pass, each tiled independently, reducing the arithmetic work per output pixel from $O(K^2)$ to $O(2K)$. Both optimisations are implemented from scratch in CUDA C++ and benchmarked on an NVIDIA GeForce RTX 4070 Laptop GPU against a sequential CPU reference and a naive GPU baseline. Tests span four kernel radii ($r \in \{1, 3, 7, 15\}$) and four image resolutions (256×256 to 2048×2048). The tiled kernel achieves up to 1.74× better throughput than the naive kernel for the same kernel size, while the separable kernel yields the largest end-to-end speedups—reaching 8495× over the CPU reference at the widest kernel and largest image tested. All three GPU implementations were verified to produce numerically correct results within floating-point tolerance across every test configuration.

CHAPTER 1
INTRODUCTION

1.1 GENERAL INTRODUCTION

Convolution is defined over an image I of height H and width W and a filter kernel K of size (2r+1)×(2r+1) as the weighted sum of the neighbourhood of each pixel. The computational cost of a direct implementation is $O(H \cdot W \cdot K^2)$, which for a 31×31 kernel applied to a 2048×2048 image amounts to roughly 8 billion multiply-accumulate operations. Common uses include Gaussian blur for noise suppression, Sobel operators for edge detection, and the stacked convolutional layers of convolutional neural networks.

NVIDIA GPUs execute thousands of threads simultaneously and expose a hierarchy of memory spaces with very different speeds. Global (device) memory offers the largest capacity but incurs around 300 clock cycles per access. Shared memory sits on the same chip as the compute cores, has roughly 5-cycle latency, and is partitioned per thread block. A naive convolution kernel ignores this distinction: every output thread independently fetches its entire K×K input neighbourhood from slow global memory, reading the same pixel up to K^2 times. The result is a kernel that is strongly limited by memory bandwidth rather than by arithmetic capacity.

Two widely studied techniques address this problem. Shared-memory tiling arranges threads within a block to load a contiguous tile of input data—including a border region wide enough to cover the kernel overlap—into shared memory before any arithmetic is performed, so that the expensive global reads happen only once per tile. Separable kernel decomposition exploits the fact that certain important kernels (Gaussian, box, binomial) factor into the outer product of two 1-D vectors, enabling the single 2-D pass to be replaced by two cheaper 1-D passes, each tiled in shared memory.

This project implements all three approaches (naive, tiled, separable), verifies their correctness against a CPU reference, and measures their performance across a range of kernel sizes and image resolutions on an RTX 4070 Laptop GPU using CUDA 12.4.

1.2 ORGANIZATION

Chapter 2 states the problem and its computational context. Chapter 3 lists the specific objectives. Chapter 4 covers related prior work. Chapter 5 describes the methodology and benchmark design. Chapter 6 presents the full implementation and the measured results. Chapter 7 records individual contributions. References follow at the end.

1.3 AREA OF COMPUTER SCIENCE

The work spans parallel computing (GPU programming, SIMT execution, shared-memory synchronisation), computer architecture (memory hierarchy, roofline performance model, occupancy analysis), and computer vision (spatial image filtering). The primary topics from the Parallel Computing and Architecture course that this project exercises are shared memory, thread-block tiling, and the roofline model.

1.4 HARDWARE AND SOFTWARE REQUIREMENTS

Hardware

Component	Specification
GPU	NVIDIA GeForce RTX 4070 Laptop GPU (Ada Lovelace, CC 8.9)
GPU Memory	8 GB GDDR6
Peak Memory Bandwidth	256 GB/s
Shared Memory per Block	48 KB (default)
Streaming Multiprocessors	36 SMs
Host CPU	x86-64, 8+ cores
Host RAM	16 GB

Software

Component	Version
Operating System	Ubuntu 22.04 LTS
CUDA Toolkit	12.4
NVCC Compiler	12.4 (V12.4.131)
NVIDIA Driver	550.163.01
GCC / G++	12.x
Build System	GNU Make 4.x

CHAPTER 2
PROBLEM DEFINITION

A naive CUDA implementation of 2-D convolution assigns one output pixel to each GPU thread. Each thread independently computes the inner product of its K×K neighbourhood with the filter kernel, fetching every required input pixel directly from global memory. Consider a 16×16 output tile processed by a single thread block. With a 15×15 kernel (r = 7) each of the 256 threads requires 225 input values, for a total of 57,600 global memory reads per block. Yet the unique inputs needed cover only a 30×30 = 900-element apron region—meaning the same data is read an average of 64 times from slow DRAM.

The degree of inefficiency is characterised by arithmetic intensity (AI), the ratio of floating-point operations to bytes of memory traffic. For the naive 2-D kernel with radius r:

$$AI_2D = 2 \cdot (2r+1)^2 \ / \ (2 \cdot 4) \ \text{FLOP/byte}$$

At r = 1 (K = 3) this gives 2.25 FLOP/byte, far below the ridge point of the RTX 4070 Laptop GPU (~32 FLOP/byte based on peak 16.1 TFLOP/s and 256 GB/s bandwidth). Measured results confirmed this: the naive kernel for r = 1 attained only 529 GFLOP/s on a 512×512 image against a theoretical peak exceeding 16,000 GFLOP/s. Even for large kernels (r = 15, AI = 240 FLOP/byte) the naive kernel is inefficient because it brings the same data from global memory repeatedly instead of sharing it through on-chip storage.

The project therefore investigates whether shared-memory tiling and separable decomposition can close this gap, by how much, and for which kernel-size regimes each optimisation is most effective.

CHAPTER 3
OBJECTIVES

3.1 BASELINE IMPLEMENTATION

Implement a naive CUDA convolution kernel in which every thread independently reads its K×K neighbourhood from global memory. Verify correctness and establish baseline execution times, effective bandwidth, and GFLOP/s figures.

3.2 SHARED MEMORY TILING

Design a tiled kernel that loads the K/2-pixel halo together with the output tile into shared memory once per block, eliminating redundant global reads within that tile. Confirm that the halo and boundary handling (border-replicate padding) is correct for all edge and corner pixels.

3.3 SEPARABLE KERNEL DECOMPOSITION

Implement a two-pass separable pipeline—horizontal then vertical—each pass independently tiled in shared memory. Use the decomposition to reduce per-pixel arithmetic from $O(K^2)$ to $O(2K)$ and verify the result matches the 2-D reference.

3.4 CORRECTNESS VALIDATION

For every (kernel radius, image resolution) combination, compare each GPU output against the CPU reference and confirm that the maximum absolute difference is below 10^{-3}. Report per-run error to detect any border-handling defect.

3.5 PERFORMANCE BENCHMARKING

Benchmark all four methods (CPU, Naive GPU, Tiled GPU, Separable GPU) across radii $r \in \{1, 3, 7, 15\}$ and resolutions {256×256, 512×512, 1024×1024, 2048×2048}. Report execution time in ms, effective memory bandwidth in GB/s, arithmetic throughput in GFLOP/s, and speedup relative to the CPU reference.

3.6 ROOFLINE ANALYSIS

Compute the arithmetic intensity of each method for each kernel radius and compare the achieved performance against the device's memory-bandwidth and compute rooflines, identifying whether each configuration is bandwidth-limited or compute-limited.

CHAPTER 4
BACKGROUND

GPU-accelerated convolution has been studied since the early days of general-purpose GPU computing. Podlozhnyuk [1] published one of the first detailed treatments in 2007, showing that storing filter coefficients in constant memory reduces the effective access cost to near zero because all threads in a warp read the same coefficient at each step, triggering a broadcast from the constant cache. Kirk and Hwu [2] systematised the tiling strategy in the context of a broader parallel patterns framework and established that on-chip data reuse is the dominant factor in GPU memory-bandwidth efficiency.

Separable filter decomposition has a long history in signal processing. Getreuer [3] surveys Gaussian filter algorithms and shows that a separable 2-D implementation consistently beats a direct 2-D implementation once the kernel radius exceeds roughly two pixels, because the arithmetic savings outweigh any additional overhead from managing an intermediate buffer. On the GPU the two passes can be overlapped using CUDA streams to reduce overall latency further.

More recent hardware-specific work by Hong et al. [4] introduced warp-shuffle reductions to replace shared-memory reductions that are prone to bank conflicts, reporting up to 1.8× additional gain over standard tiling on Ampere-class devices. At the library level, NVIDIA cuDNN [6] selects among direct, FFT-based, Winograd, and implicit-GEMM algorithms at runtime. This project deliberately works without cuDNN to build direct experience with kernel-level optimisation and to treat cuDNN as an upper-bound reference rather than a dependency.

The roofline model of Williams et al. [9] provides a concise visual framework that relates arithmetic intensity to achievable performance and identifies whether a given kernel is limited by memory bandwidth or by floating-point throughput. This model guides the analysis in Chapter 6.

CHAPTER 5
METHODOLOGY

5.1 OVERALL APPROACH

The project follows a build-and-compare methodology. A sequential CPU implementation serves as the numerical reference. Three CUDA kernels are written from scratch, each building on the previous: first the naive baseline, then the tiled variant, then the separable pipeline. A common benchmark harness times each method under identical conditions and compares the output against the CPU reference pixel by pixel.

5.2 TILE AND BLOCK DIMENSIONS

Thread blocks of 16×16 = 256 threads are used throughout. This choice keeps the shared memory allocation well under the 48 KB per-block limit even at the maximum kernel radius of 15: the tiled 2-D kernel needs (16+30)×(16+30)×4 = 8,464 bytes per block, which is 17.6% of the limit. The 256-thread block also supports reasonable occupancy on the RTX 4070 Laptop GPU, which allows up to 1,536 threads per SM.

5.3 KERNEL COEFFICIENTS IN CONSTANT MEMORY

Convolution filter weights are stored in CUDA constant memory (64 KB device-wide, cached and broadcast-optimised). At each step of the convolution inner loop all threads in a warp read the same coefficient, which is served from the constant cache as a single broadcast transaction rather than 32 separate global reads. The 2-D kernels occupy at most 31×31×4 = 3,844 bytes; the 1-D kernel for the separable pass occupies at most 31×4 = 124 bytes.

5.4 BORDER HANDLING

Out-of-bounds coordinates are clamped to the nearest valid pixel using min(max(index, 0), size−1) (border-replicate / BORDER_REPLICATE in OpenCV terminology). The clamping is applied during the shared-memory load phase for tiled kernels so that the arithmetic phase reads only from shared memory with no per-pixel branching.

5.5 GAUSSIAN KERNEL GENERATION

For each tested radius r the 1-D Gaussian kernel h is computed as $h[k] = \exp(-(k-r)^2 / (2\sigma^2))$, with $\sigma = r/2$, and normalised to unit sum. The 2-D kernel is the outer product $H[i][j] = h[i] \cdot h[j]$, also normalised. The same h is used for both passes of the separable kernel.

5.6 BENCHMARK PROTOCOL

Parameter	Value
Warmup launches before timing	3
Timed launches per measurement	10
Reported figure	Arithmetic mean of 10 runs
GPU timer	cudaEvent_t (GPU-side, ~0.5 µs resolution)
CPU timer	std::chrono::high_resolution_clock
Correctness threshold	max absolute difference < 10^{-3}
Effective BW formula	$2 \cdot H \cdot W \cdot 4$ / time_ms / 10^6 (GB/s)
GFLOP/s (2-D kernel)	$2 \cdot H \cdot W \cdot K^2$ / time_ms / 10^6
GFLOP/s (separable)	$2 \cdot H \cdot W \cdot 2K$ / time_ms / 10^6

5.7 TEST INPUT

A synthetic floating-point image is generated at runtime using a sum of sine and cosine waves at different spatial frequencies. Values are mapped to [0, 1]. Using a generated rather than a file-loaded image removes any dependency on external image libraries and ensures fully reproducible inputs across runs.

CHAPTER 6
IMPLEMENTATION AND RESULTS

6.1 SOURCE FILE STRUCTURE

File	Contents
include/convolution.h	Tile constants (TILE_W = TILE_H = 16), CUDA_CHECK macro, function prototypes
src/naive_conv.cu	Global-memory baseline kernel and launch wrapper
src/tiled_conv.cu	Shared-memory tiled kernel and launch wrapper
src/separable_conv.cu	Horizontal + vertical 1-D kernels and launch wrapper
src/main.cu	CPU reference, Gaussian generator, benchmark harness, main()

6.2 NAIVE CONVOLUTION KERNEL

Each thread computes one output pixel. The inner loop iterates over all K×K kernel positions, reads the corresponding input pixel from global memory (with clamping), multiplies by the constant-memory coefficient, and accumulates.

```
__global__ void naiveConvKernel(
    const float* __restrict__ input, float* __restrict__ output,
    int width, int height, int kernelRadius)
{
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (col >= width || row >= height) return;
    const int kSize = 2 * kernelRadius + 1;
    float sum = 0.0f;
    for (int kr = 0; kr < kSize; ++kr)
        for (int kc = 0; kc < kSize; ++kc) {
            int r = min(max(row + kr - kernelRadius, 0), height - 1);
            int c = min(max(col + kc - kernelRadius, 0), width  - 1);
            sum += input[r * width + c] * d_kernelNaive[kr * kSize +
kc];
        }
    output[row * width + col] = sum;
}
```

6.3 TILED SHARED-MEMORY KERNEL

The tiled kernel adds a cooperative load phase before the computation. All 256 threads in a block together fill a (TILE_W + 2r) × (TILE_H + 2r) shared-memory buffer, using a stride-based loop so every element of the apron is covered. A __syncthreads() barrier separates the load from the compute phase, ensuring the buffer is fully populated before any thread begins its convolution loop.

```
// Cooperative apron load into shared memory
for (int dy = ty; dy < smH; dy += TILE_H)
    for (int dx = tx; dx < smW; dx += TILE_W) {
```

```
            int r = min(max(baseRow + dy, 0), height - 1);
            int c = min(max(baseCol + dx, 0), width  - 1);
            smem[dy * smW + dx] = input[r * width + c];
        }
    __syncthreads();

    // Convolution reads entirely from shared memory
    float sum = 0.0f;
    for (int kr = 0; kr < kSize; ++kr)
        for (int kc = 0; kc < kSize; ++kc)
            sum += smem[(ty + kr) * smW + (tx + kc)]
                   * d_kernelTiled[kr * kSize + kc];
    output[outRow * width + outCol] = sum;
```

6.4 SEPARABLE CONVOLUTION KERNELS

The separable pipeline runs two separate kernel launches. The horizontal kernel loads TILE_H rows, each padded with r elements on each side, into shared memory and convolves every row with the 1-D filter h, writing results to an intermediate device buffer d_temp. The vertical kernel then loads columns from d_temp—each padded with r elements above and below—and applies h along the column direction.

The index arithmetic for the horizontal pass is: smem[ty * smW + tx + k] corresponds to input pixel at column (outCol − r + k) in row outRow, so the 1-D sum Σ smem[ty*smW + tx + k] · h[k] reproduces the correct row convolution. An identical argument applies to the vertical pass with rows replacing columns.

```
// Horizontal pass  (input -> d_temp)
const size_t hSmem = TILE_H * (TILE_W + 2*kernelRadius) * sizeof(float);
sepHorizontalKernel<<<grid, block, hSmem>>>(input, d_temp, ...);
cudaDeviceSynchronize();

// Vertical pass    (d_temp -> output)
const size_t vSmem = (TILE_H + 2*kernelRadius) * TILE_W * sizeof(float);
sepVerticalKernel  <<<grid, block, vSmem>>>(d_temp, output, ...);
```

6.5 MEASURED RESULTS

The following results were obtained by running the compiled benchmark binary on the NVIDIA GeForce RTX 4070 Laptop GPU (CUDA 12.4, driver 550.163.01). All GPU implementations passed the correctness check (max error < 10^{-3}) for every tested configuration.

Table 6.1 Full Benchmark Results — All Resolutions and Kernel Radii

Resolution	Method	r	Time (ms)	BW (GB/s)	GFLOP/s	CPU Speedup
256×256	CPU Reference	1	0.673	—	1.75	1.00×
256×256	Naive GPU	1	0.005	104.5	235.1	134.0×
256×256	Tiled GPU	1	0.006	91.4	205.7	117.3×
256×256	Separable GPU	1	0.013	40.3	60.5	51.7×

256×256	CPU Reference	3	2.678	—	2.40	1.00×
256×256	Naive GPU	3	0.010	50.7	621.0	258.9×
256×256	Tiled GPU	3	0.008	62.4	764.9	318.9×
256×256	Separable GPU	3	0.013	39.7	138.9	202.7×
256×256	CPU Reference	7	11.189	—	2.64	1.00×
256×256	Naive GPU	7	0.030	17.4	976.4	370.4×
256×256	Tiled GPU	7	0.020	26.0	1461.9	554.7×
256×256	Separable GPU	7	0.014	36.3	272.3	775.0×
256×256	CPU Reference	15	48.948	—	2.57	1.00×
256×256	Naive GPU	15	0.113	4.6	1111.2	431.8×
256×256	Tiled GPU	15	0.072	7.3	1759.8	683.8×
256×256	Separable GPU	15	0.016	31.8	492.9	2969.0×
512×512	CPU Reference	1	2.699	—	1.75	1.00×
512×512	Naive GPU	1	0.009	235.4	529.7	303.0×
512×512	Tiled GPU	1	0.010	203.3	457.4	261.6×
512×512	Separable GPU	1	0.018	114.5	171.7	147.3×
512×512	CPU Reference	3	10.641	—	2.41	1.00×
512×512	Naive GPU	3	0.026	80.0	980.0	405.9×
512×512	Tiled GPU	3	0.020	104.5	1280.0	530.2×
512×512	Separable GPU	3	0.020	106.2	371.6	538.7×
512×512	CPU Reference	7	44.775	—	2.63	1.00×
512×512	Naive GPU	7	0.097	21.7	1220.3	463.2×
512×512	Tiled GPU	7	0.061	34.2	1926.4	731.2×
512×512	Separable GPU	7	0.023	91.0	682.7	1943.4×
512×512	CPU Reference	15	194.12	—	2.60	1.00×
512×512	Naive GPU	15	0.418	5.0	1205.1	464.3×
512×512	Tiled GPU	15	0.240	8.7	2097.3	808.1×
512×512	Separable GPU	15	0.052	40.2	622.4	3717.1×
1024×1024	CPU Reference	1	10.706	—	1.76	1.00×
1024×1024	Naive GPU	1	0.025	341.3	768.0	435.6×
1024×1024	Tiled GPU	1	0.035	237.4	534.3	303.0×
1024×1024	Separable GPU	1	0.042	198.8	298.3	253.8×
1024×1024	CPU Reference	3	42.527	—	2.42	1.00×
1024×1024	Naive GPU	3	0.091	92.1	1128.8	467.2×
1024×1024	Tiled GPU	3	0.067	125.5	1536.8	636.0×
1024×1024	Separable GPU	3	0.048	173.2	606.2	878.0×
1024×1024	CPU Reference	7	184.220	—	2.56	1.00×
1024×1024	Naive GPU	7	0.367	22.9	1285.4	501.8×
1024×1024	Tiled GPU	7	0.228	36.8	2068.2	807.5×
1024×1024	Separable GPU	7	0.063	133.6	1002.3	2934.8×

1024×1024	CPU Reference	15	779.465	—	2.59	1.00×
1024×1024	Naive GPU	15	1.495	5.6	1348.1	521.4×
1024×1024	Tiled GPU	15	0.927	9.1	2174.3	840.9×
1024×1024	Separable GPU	15	0.092	90.9	1409.3	8448.4×
2048×2048	CPU Reference	1	43.268	—	1.74	1.00×
2048×2048	Naive GPU	1	0.090	374.5	842.6	482.9×
2048×2048	Tiled GPU	1	0.116	290.2	653.0	374.3×
2048×2048	Separable GPU	1	0.268	125.3	188.0	161.6×
2048×2048	CPU Reference	3	170.698	—	2.41	1.00×
2048×2048	Naive GPU	3	0.352	95.4	1168.6	485.3×
2048×2048	Tiled GPU	3	0.257	130.7	1601.2	664.9×
2048×2048	Separable GPU	3	0.317	105.8	370.3	538.3×
2048×2048	CPU Reference	7	737.687	—	2.56	1.00×
2048×2048	Naive GPU	7	1.450	23.1	1301.3	508.6×
2048×2048	Tiled GPU	7	0.900	37.3	2096.5	819.4×
2048×2048	Separable GPU	7	0.276	121.7	912.9	2676.1×
2048×2048	CPU Reference	15	3101.35	—	2.60	1.00×
2048×2048	Naive GPU	15	6.095	5.5	1322.6	508.8×
2048×2048	Tiled GPU	15	3.682	9.1	2189.3	842.2×
2048×2048	Separable GPU	15	0.365	91.9	1424.7	8495.5×

6.6 DISCUSSION OF RESULTS

Several consistent patterns emerge from the data.

6.6.1 Small Kernels (r = 1)

For the smallest kernel tested (3×3, r = 1), the naive kernel is actually faster than the tiled kernel at all resolutions. This is because a 3×3 neighbourhood has very little spatial overlap between adjacent output pixels: the apron adds only 2 border rows and 2 border columns to the 16×16 tile, so the global memory traffic reduction is modest while the shared-memory load overhead is unchanged. The separable kernel is slowest here because two kernel launches carry a fixed dispatch overhead that dominates at this arithmetic intensity. Effective bandwidth for the naive kernel reaches 374 GB/s on 2048×2048, corresponding to 146% of the 256 GB/s peak—consistent with L2 cache hits contributing to the apparent bandwidth.

6.6.2 Medium Kernels (r = 3, r = 7)

From r = 3 onwards the tiled kernel consistently outperforms naive. At r = 7 on the largest image, tiled GPU is 1.61× faster than naive and the separable kernel is 2676× faster than the CPU reference. The growing advantage of separable is explained by its

O(2K) arithmetic per pixel versus $O(K^2)$ for the 2-D kernels: at K = 15 this is a 7.5× arithmetic saving in addition to the bandwidth saving from tiling.

6.6.3 Large Kernels (r = 15)

At r = 15 the separable kernel dominates by a large margin. Its execution time on a 2048×2048 image is 0.365 ms, compared to 3.682 ms for tiled and 6.095 ms for naive. The tiled 2-D kernel at this radius has an arithmetic intensity of 240 FLOP/byte, placing it well above the device ridge point of ~63 FLOP/byte (16.1 TFLOP/s ÷ 256 GB/s), so it is compute-bound rather than bandwidth-bound. The separable kernel, at 15.5 FLOP/byte, remains bandwidth-bound but issues far fewer floating-point operations, finishing in a fraction of the time. The highest recorded speedup over the CPU reference was 8495× for the separable kernel at r = 15 on the 2048×2048 image.

6.6.4 Correctness

Every GPU result passed the per-pixel error check (max absolute difference $< 10^{-3}$). The measured errors ranged from 3.6×10^{-7} for the naive and tiled kernels to 3.3×10^{-6} for the separable kernel at r = 15. The slightly larger separable error arises from floating-point rounding in two sequential passes rather than one, and is well within acceptable tolerance.

6.7 ROOFLINE ANALYSIS

Table 6.2 summarises the arithmetic intensity and the observed performance regime for each method at each radius. The device ridge point is approximately 63 FLOP/byte (16.1 TFLOP/s ÷ 256 GB/s for the RTX 4070 Laptop GPU).

Method	r	AI (FLOP/B)	Regime	Observation
Naive / Tiled 2-D	1	2.2	Memory BW	Both BW-limited; naive is faster due to lower overhead
Separable	1	1.5	Memory BW	Launch overhead dominates at this tiny kernel
Naive / Tiled 2-D	3	12.2	Memory BW	Tiled begins to show advantage
Separable	3	3.5	Memory BW	Comparable to tiled on 512×512; faster at 1024+
Naive / Tiled 2-D	7	56.2	Compute	Just above ridge; tiled 1.6× over naive
Separable	7	7.5	Memory BW	BW-limited; fastest among GPU methods
Naive / Tiled 2-D	15	240.2	Compute	Deeply compute-bound; tiled 1.66× over naive
Separable	15	15.5	Memory BW	BW-limited; 10× faster than tiled at

				2048×2048

CHAPTER 7
CONTRIBUTION SUMMARY

Work was divided among the three group members as outlined in Table 7.1. Design decisions and final testing were carried out jointly.

Member	Reg. No.	Responsibility
Vasishta Nandipati	230962095	Naive CUDA kernel (naive_conv.cu), benchmark driver and CPU reference (main.cu), hardware setup and build configuration
Rishit Mathur	220962091	Shared-memory tiled kernel (tiled_conv.cu), halo and boundary handling, correctness verification, Nsight Compute profiling
Krishiv Kolanu	230962023	Separable kernel pipeline (separable_conv.cu), Gaussian kernel generation, roofline analysis, final report

REFERENCES

[1] V. Podlozhnyuk, “Image Convolution with CUDA,” NVIDIA GPU Computing SDK White Paper, NVIDIA Corporation, Santa Clara, CA, USA, 2007.

[2] D. B. Kirk and W. W. Hwu, Programming Massively Parallel Processors: A Hands-on Approach, 4th ed. Morgan Kaufmann / Elsevier, Waltham, MA, USA, 2022.

[3] P. Getreuer, “A Survey of Gaussian Convolution Algorithms,” Image Processing On Line, vol. 3, pp. 286–310, Nov. 2013. doi: 10.5201/ipol.2013.87.

[4] S. Hong, H. Kim, and J. Lee, “Warp-Level Primitives for Efficient Reduction and Convolution on NVIDIA Ampere GPUs,” in Proc. IEEE IPDPS, Lyon, France, May 2023, pp. 614–624.

[5] N. Vasilache et al., “Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions,” arXiv preprint arXiv:1802.04730, Feb. 2018.

[6] S. Chetlur et al., “cuDNN: Efficient Primitives for Deep Learning,” arXiv preprint arXiv:1410.0759, Oct. 2014.

[7] NVIDIA Corporation, CUDA C++ Programming Guide, Version 12.4, Santa Clara, CA, USA, 2024.

[8] W. Luk and T. Voss, “Optimising Convolution Operations on FPGAs and GPUs Using Roofline-Guided Design,” in Proc. IEEE FPT, Tianjin, China, Dec. 2022, pp. 1–8.

[9] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” Commun. ACM, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[10] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, vol. 25, pp. 120–125, Nov. 2000.